

G14
Snowledge

Final Report

Micko Hakala
Sanni Onkalo
Joonas Arola
Martin Pochat-Cottilloux
Piia Lehtinen
Santeri Virtanen

Version history

Version	Date	Author	Description
0.1	13.11.2022	Sanni Onkalo	First draft
0.2	15.11.2022	Micko Hakala	Modify 3.1, 3.3
0.3	16.11.2022	Micko Hakala	Add 5.2, 5.3, 10
0.4	16.12.2022	Sanni Onkalo	Finalize the document

Contents

1	Introduction.....	4
1.1	Purpose of the report	4
1.2	Product and environment.....	4
1.3	Definitions, abbreviations and acronyms.....	4
2	Project organisation	4
2.1	Group.....	4
2.2	Customer	5
2.3	Other stakeholders.....	5
3	Project implementation	5
3.1	Communication	5
3.2	Tools and technologies	6
3.3	Sprints	6
3.4	Deliverables and outcomes	7
3.5	Restrictions and limitations	7
3.6	Third party components, licenses and IPRs.....	7
3.7	GDPR, ePrivacy and related matters	6
4	Working hours	8
4.1	Personal contributions to project	8
4.2	Peer feedback	9
5	Quality assurance	9
5.1	General description of testing	9
5.2	Bug reporting.....	10
5.3	Conclusions on product's quality	10
6	Risks and problems.....	10
6.1	Foreseen risks.....	10
6.2	Risks not foreseen	11
7	Not implemented in this project	11
7.1	Rejected ideas	11
7.2	Further development.....	11
8	Lessons learnt.....	12
9	Comments about the course.....	12
10	Statistics	14

1 Introduction

1.1 Purpose of the report

This is a Final report for Project Snowledge that took place in Autumn 2022. This document describes the outcomes of the project, how the project performed and what the team learned from the project.

1.2 Product and environment

Snowledge is an outdoor application produced by Pallaksen Pöllöt. The application offers information on current snow and weather conditions in Pallas area, together with a Rescue element for users in emergency. Typical application user is skier or other explorer in Pallas area who wants to know about the environment and conditions in the area.

The Rescue element is only present in the mobile application of the product. It is a GPS location feature that helps National Rescue Service personnel locate victims of an accident. Application user can send a help request from the mobile application and the application will notify other users nearby, personnel of Pallaksen Pöllöt and National Rescue Service about the incident.

1.3 Definitions, abbreviations, and acronyms

Dev	Developer working on Snowledge
End-user	People in Pallas that are interested snow and weather conditions
PB	Product backlog
PM	Project manager
PO	Product owner
SB	Sprint backlog
SCRUM	Method for project management (Scrum guide, 2020)
Super-user	Admin user
UI	User interface
MMT	Metrics Monitoring Tool

2 Project organisation

2.1 Group

PM/PO/Senior developer Micko Hakala, micko.hakala@tuni.fi, 0451216992
Project management
Weekly reports (Monday on every odd week)
Communication with the customer
Lead on meetings
Code reviews
Testing

PM/PO Sanni Onkalo, sanni.onkalo@tuni.fi, 0445959231

Project management

Weekly reports (Monday on every even week)

Meeting invitations and agendas

Meeting notes

Code reviews

Testing

Developer, Joonas Arola, joonas.arola@tuni.fi, 0408233463.

UI Design

Implementation

Developer, Santeri Virtanen, santeri.virtanen@tuni.fi, 0442115499.

UI Design

Implementation

Developer, Martin Pochat-Cottilloux, martin.pochat-cottilloux@tuni.fi, +33640147241

UI Design

Implementation

Developer, Piia Lehtinen, piia.lehtinen@tuni.fi, 0505549977

UI Design

Implementation

2.2 Customer

Customer company, Pallaksen Pöllöt.

Primary contact, Juuso Holstein, juuso@pallaksenpollot.com

Secondary contact, Arto Hämäläinen, artoahamalainen@gmail.com

2.3 Other stakeholders

Coach, Pekka Mäkiäho, pekka.makiaho@istekki.fi

3 Project implementation

3.1 Communication

Team had regular weekly meetings on Teams and occasionally on campus. There were also weekly meetings with the customer on Teams. Team had biweekly sprint review meetings with the coach and the customer also on Teams.

For internal communication the team used Telegram group messaging. Email was used to communicate with the customer and the coach.

There was also a group from Oulu university that worked with the project and the team used Discord to communicate with them.

3.2 Tools and technologies

Gitlab was used for version control, issue management and Kanban board. This tool was chosen because it was familiar to the team members, and it offers version control and issue management in the same place.

MMT was used for weekly reporting, working hours logging and risk management. This tool is the official tool used in the course.

Flutter is the framework used to create the mobile application. This framework is based on **Dart**, which is a programming language optimised for multi-platform applications.

3.3 Sprints

The project consisted of 7 sprints in total, requirements gathering (0), five implementation sprints (1-5), and quality assurance (6). Each sprint was two weeks long apart from the last one (6) lasting only a week.

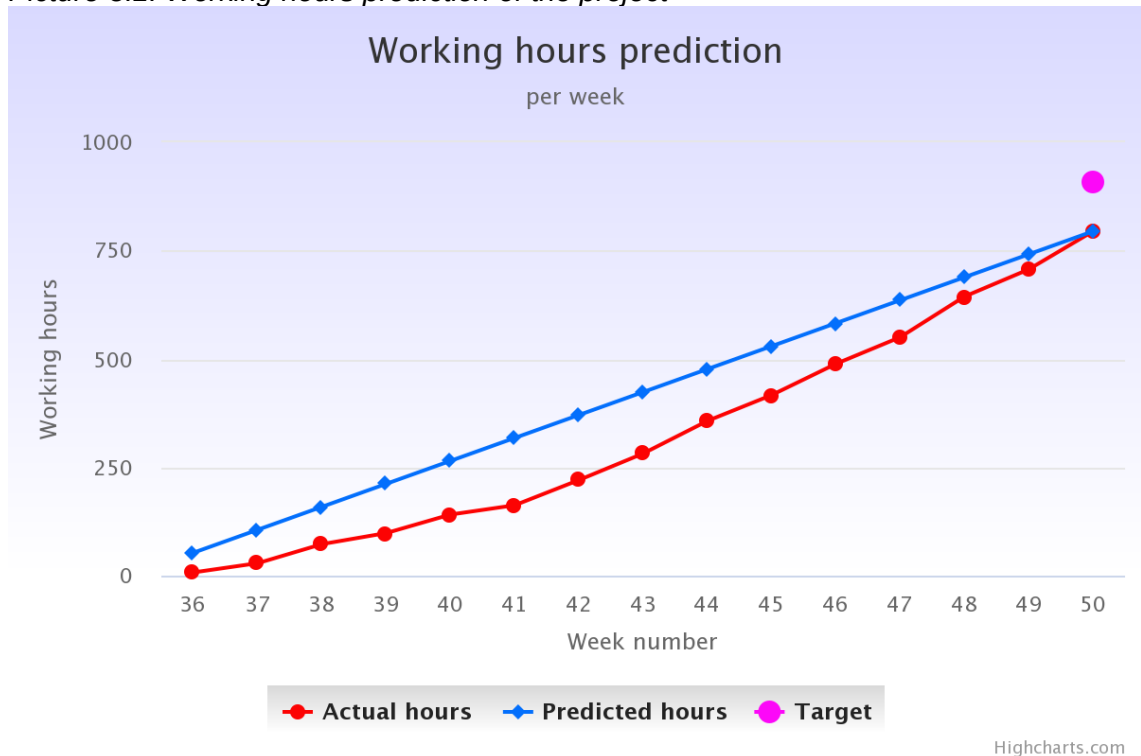
During this project, total of 55 task were implemented and 7 rejected. Project managers updated the backlog during the project as designs were clear. There were few requirements from the customer that team know at start of this project that probably won't get to the final product, so project managers did not add those to the backlog.

There is no data about how many tasks were not implemented during a certain sprint because team moved all tasks that were still in "Doing" when sprint ended to the next sprint. Backlog prioritisation worked well, and all the important features were implemented.

Here is a list that how many tasks were implemented during specific sprint:

Sprint 0, 0 tasks
Sprint 1, 4 tasks
Sprint 2, 4 tasks
Sprint 3, 9 tasks
Sprint 4, 8 tasks
Sprint 5, 15 tasks
Sprint 6, 14 tasks

Picture 3.1. Working hours prediction of the project



3.4 Deliverables and outcomes

The product is a finished app that is released to the app stores. The customer owns the rights to the application and the Gitlab repository containing all the source files related to the project.

The state of the product is finished but ready for further development as the customers sees necessary.

Total of 4560 lines of source code in a project folder we mainly worked with. Estimation of the new lines of codes is around 3000. Count of classes and files is 26.

3.5 Restrictions and limitations

There was a plan to make a tutorial when first opening the app to help the users get familiar with the app. This was rejected due to bugs and the current structure of the app didn't allow it to be implemented well enough to be included in the app. It also didn't prove to be that important of a feature.

3.6 Third party components, licenses and IPRs

The app uses Maptiler services for maps.

Customer has all rights to the product and its further development.

4 Working hours

Table 4.1. Realized working hours in person-hours by person and task.

	Micko	Sanni	Joonas	Santeri	Martin	Piia	Total
Documentation	20.5	28.2	1.5	4.5	4.25	8.75	65.7
Requirements	4	1.5	0	0	0	0	5.5
Design	0.5	6.25	2	3.5	1	9	22.25
Implementation	45.5	22.45	83.5	68.25	62.25	59.2	341.15
Testing	0	6.7	3	1	3	0	13.2
Meetings	31	29.45	22.5	25.2	26.75	23.25	158.15
Studying	18.5	16.5	2.5	3	14	19.5	75
Other	5	12.5	21	12	6	0.5	57
Lectures	6.25	11.95	8.5	11.75	14	5.5	62.45
Total	132.25	135.5	144.5	129.2	131.25	125.7	795.9

Table 4.2. Realized working hours (weekly totals / project).

	Weekly total
Week 36	8
Week 37	21
Week 38	44
Week 39	26
Week 40	43
Week 41	21
Week 42	58
Week 43	61
Week 44	75
Week 45	60
Week 46	72
Week 47	62
Week 48	94
Week 49	63
Week 50	108
Total	816

4.1 Personal contributions to project

Micko Hakala worked mainly as a project manager with Sanni Onkalo. Responsibilities included planning issues, lead on meetings, communication with customer, code reviews and testing. Micko took responsibility from backend side of the product and took care of bringing the application to the app store. He did also some work at the server when there were some issues when updating the web application with the new paths.

Sanni Onkalo worked as a project manager with Micko Hakala. Responsibilities included planning issues and sprints, code reviews, planning meeting agendas and returning course related documents. She planned the use cases and test cases, and designed action diagrams for Rescue element. She also worked as a testing manager, initialized Codemagic pipeline and performed system testing. In addition to project management tasks, she implemented a few features in the frontend, including hamburger menu functionality.

Santeri Virtanen worked on the UI designs at the beginning of the project and then implemented the first views of the app and user information view. He also worked on various other tasks during the project.

Piia Lehtinen participated in the application's UI design presenting several ideas with prototypes from the beginning to the end of the project. The client liked many of the ideas and gave the green light to their implementation. In addition to visual implementations she also implemented a few functionalities in the application, such as the hamburger menu (side menu), its first version.

Joonas Arola started with UI designing with the team and was responsible for most of the code related to maps and location usage. He also implemented various other stuff ranging from rescue element dialogs to server communications. He worked both on the backend and frontend side of the application and during the late phases of the project he worked mostly on fixing bugs and testing the application.

Martin Pochat-Cottilloux worked as a developer during this project. He took several tasks related to the front end (implementing dialogs, implementing the redesign of the first views). He worked on backend related tasks, for example notifying other helpers that the help request is no longer relevant and notifying users that no users is close when asking for help. He fixed several bugs (timer bug, keyboard overlay, returning to the app after going to settings). He also worked on the global state of the app to maintain between pages and has done some code restructuring to follow the best coding practices. He is responsible for publishing the app on the Google Play Store.

4.2 Peer feedback

The team's thoughts on peer feedback:

- Giving peer feedback was technically easy and simple.
- The first feedback period came perhaps too early. At that time, there wasn't much experience with the other team members (as we progressed slowly in the beginning).
- It is good to receive feedback at different stages of the project.
- In principle and perhaps in practice, feedback steers the project in the right direction and improves members' performance.
- When asked in which skills a member should improve, it is not easy to say, because everyone usually knows their own strengths and weaknesses, and they improve as the project is moving forward.
- Team also gave oral feedback to other group members during the project, mostly from jobs well done.

5 Quality assurance

5.1 General description of testing

New features were tested during implementing and project used mostly exploratory testing. The developers did the testing for the features that they implemented as a part of the task.

PMs did integration testing for the tasks before merging the code to the master branch.

Before publishing the application to the stores, testing managers conducted system testing at their local development environments and iOS test environment TestFlight. After system testing passed, team published application to the Apple App Store and so the customer was able to do acceptance testing.

Test results from system testing were documented to Test report.

5.2 Bug reporting

Total of 11 bugs were found during and before system testing and those were reported to bug database (appendix 1). Of those, 8 bugs were fixed, 1 was worked around, 1 was found to out of the scope of this project, and 1 was left open, since it awaits response from Digia.

5.3 Conclusions on product's quality

Quality of this product is good based on the testing and user experience. Flutter may not be best choice for application that tracks location, but team managed to prove a product that works.

6 Risks and problems

6.1 Foreseen risks

Picture 6.1. Project risk table

Risk	Impact	Probability
Member of the group drops out from the projec	Very High	Very Low
Member of the group gets ill (flu etc.)	Very Low	High
Lack of technical knowledge in a project team	Very High	Very Low
Changes in project specifications	High	Low
Lack of communication team - customer	Medium	Very Low
No understanding how application is done	High	Medium
Incomplete risk assessment	Low	Low
Customer wants to end project because fo time	Very High	Very Low
Lack of testing - bugs in application	High	Low
Member gets seriously ill cant continue proje	Very High	Very Low

Risk of a group member quitting realised a few weeks into the project. This didn't have too much of an impact as it happened at an early stage of the project. The group was able to adapt to the situation and take another look at the project goals to see what could still be implemented.

At the very start of the project the risk of too little communication between the customer and the group impacted the group a little. The first few weeks of the project were slow

as the project's goals weren't yet clear. In the end this didn't impact the project that much, other than the group was lacking behind in the work hours midway through the project.

One risk that also realised during the project was the adding of new requirements. This didn't affect the project negatively, but it took some time to come up and design these new features.

6.2 Risks not foreseen

A risk that we didn't see coming was the differences between the Android and the iOS versions of the application. During the project most of the developers used a PC and an Android emulator for implementing and testing the new features. At the end of the project when we started testing the whole application on both platforms a lot of bugs were found. Especially related to the location and notification side of the application. This caused a lot of work in the end.

Knowing this risk in the beginning would have saved some time and the differences between the platforms could have been considered at an earlier stage of the project.

7 Not implemented in this project

7.1 Rejected ideas

At the start of the project there were some plans on adding weather stations in the app for the Pallas region. In the end it was decided we would not go implementing it due to lack of time and the focus being on the mobile application side of the project. Implementing the weather stations would have required adding them to the web application side which we didn't have time to take a bigger look at.

There were also some plans to take the app's services to other fell areas as well. In the end this was not done for the same reasons as the weather stations, as it would have been on the web application side. This feature also was not a priority for the customer.

There was also a plan to make the app also available in English. This was rejected due to lack of time.

7.2 Further development

Further development could be to further improve the rescue element of the application. A battery saving mode could also be added that would limit the amount of charge the app uses in the background. Some more information could also be added to the markers in the rescue maps. For example, if you click on a helper's marker it could show statistics similarly to the lumisovellus.fi/rescue page.

Currently the app uses two different kinds of maps, one in the web application and one in the mobile application. Both could be made with the same map provider to make the application look more uniform.

The mobile code could also be refactored to make the architecture of the app

simpler. This could also make possible the implementation of a tutorial in the app in a more meaningful way.

It would have been good to create a similar visual appearance for the different parts of the application. This was not a requirement of the customer, but we did it in our part of the project, but there was not enough time to do it for the whole application. The maps for example differ from each other.

8 Lessons learnt

Team learned new technologies. For example, only one of the team members was familiar with Flutter before this project. Team also got more familiar with version control and working with a Kanban board. The importance of meetings and communication inside the team and between the customer became clear. Developing the mobile application and improving its visual look went well. The customer liked the new look and features.

At first the existing code and structure of the product seemed difficult to understand. It wasn't clear where to start. However, this didn't prove to be an issue as the work started and the team had time to take a further look into the code.

If this project were to start over, at the start of the project there should be more communication between the group and the customer. In the beginning there are a lot of questions about the project and many things are still unclear. Having more communication would answer these questions as quickly as possible and the project would get moving faster.

When starting work on an existing application like in this project, it's important to not get too depended on already existing code and not to be scared to implement in a completely new way.

Using a cross platform framework for an app that uses the GPS location might not be the best choice.

Communication within the group worked well. We could have been in contact with the customer more often, but the cooperation and communication went well. The course staff communicated well to keep us on track.

About forming a team and project start. It might have been effective to assign a few team members to certain tasks. For example, the visual design could have been done by two team members, so that everyone would not think about the same things and make their own solutions. On the other hand, when we thought about things together with all team members and presented solutions, the best ideas were born.

9 Comments about the course

Good thing about the course is the fact that the groups work with real customers and create something concrete.

A not so good thing is we depend on the customer's communication. If the customer is slow to react, the whole project may be at a complete halt until we get a response from the customer.

10 Statistics

Snowledge, group size of 6

Who we are



Sanni Onkalo
620 student
PM/PO, Senior developer



Micko Hakala
620 student
PM/PO, Senior developer



Piia Lehtinen
610 student
Developer



Joonas Arola
610 student
Developer

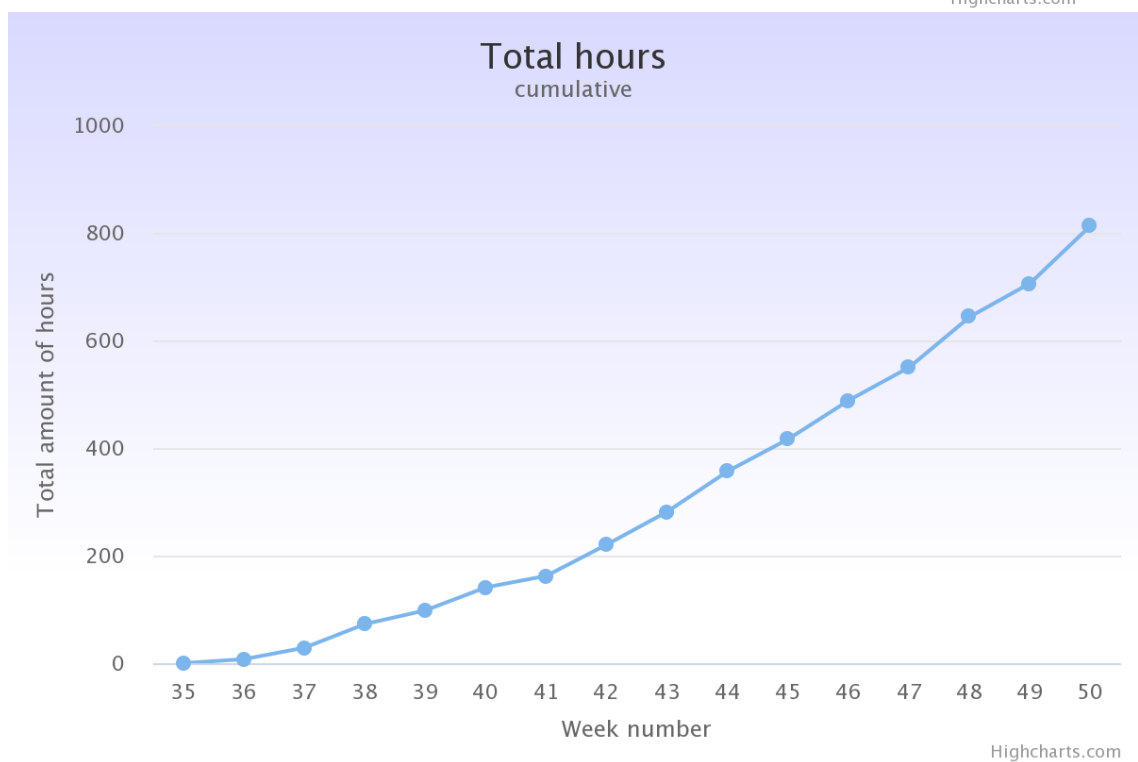
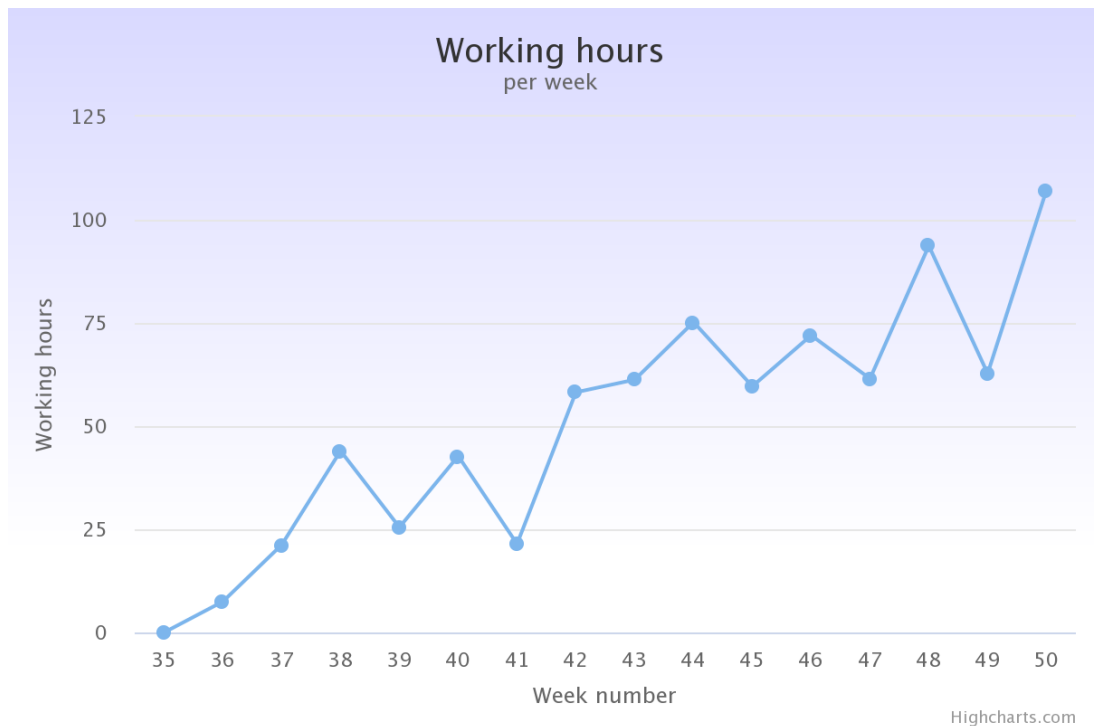


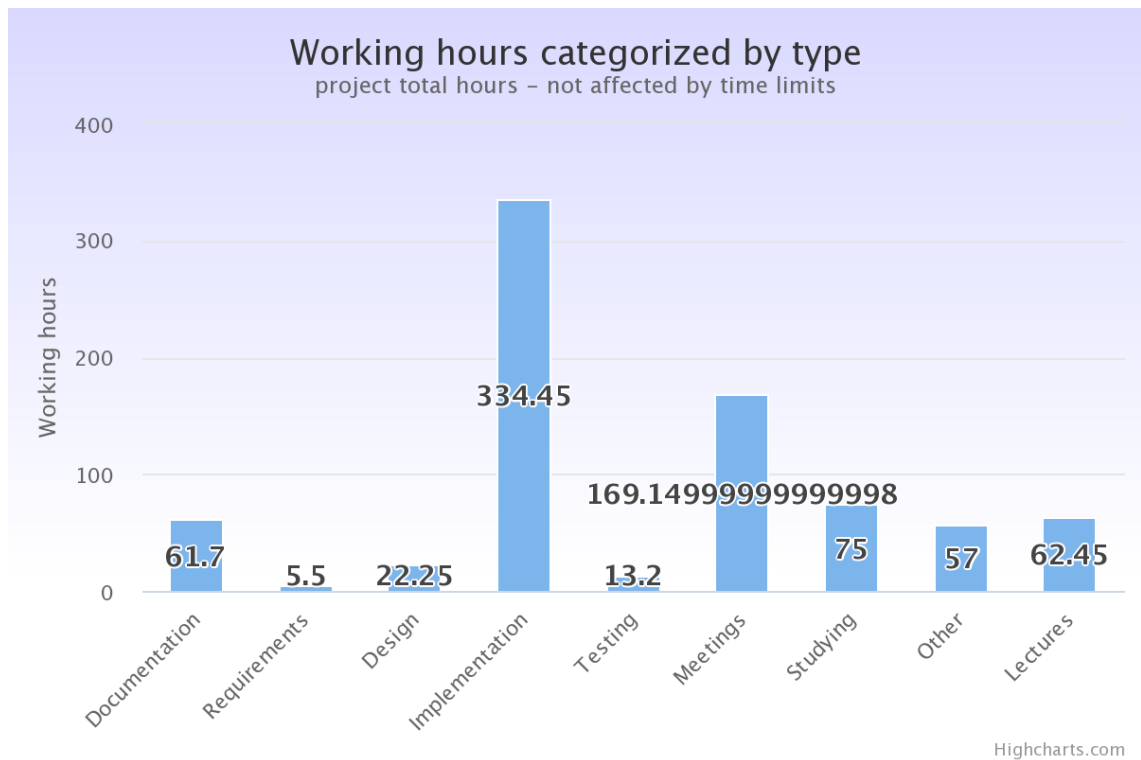
Santeri Virtanen
610 student
Developer



Martin Pochat-Cottilloux
610 student
Developer

Customer: Pallaksen pöllöt
Tools: Flutter, Dart, React, python, GitLab
Code lines: Estimation of 4500
Number of classes / files: 26
Number of revisions: 181





Number of Implemented items / rejected: 55 / 7

Learned a lot from mobile development, understood new things about project work and importance of communication with the customer.